

1. Dépannage des pipelines CI/CD

1.1 Authentification & Credentials

- **Symptôme** : Erreurs d'accès aux dépôts ou ressources (HTTP 401/403, messages authentication failed ou permission denied).
- **Causes probables** : Mauvaises clés/API tokens, variables d'environnement manquantes ou expirées, autorisations insuffisantes (ex. token sans droits sur le repo).
- **Diagnostic** : Vérifier les logs d'erreurs et messages détaillés (p.ex. dans Jenkins, dans les logs GitLab ou Azure Pipelines). Contrôler les credentials configurés : Jenkins « Credentials », variables CI/CD GitLab, GitHub Secrets, Azure Pipeline Variables/Service Connections. Tester les droits d'accès : p.ex. `kubectl auth can-i` n'est pas pour CI directement, mais à titre d'exemple pour tester les permissions d'un compte dans Kubernetes.
- **Solutions** :
 - *Jenkins* : Ajouter/mettre à jour les credentials dans l'interface Jenkins (onglet *Credentials*), et référencer leur ID dans la pipeline (p.ex. `withCredentials([usernamePassword(credentialsId: 'git-cred', ...)])` pour Git). Vérifier le *URL* et le type (SSH vs HTTP).
 - *GitLab CI* : Définir les variables CI/CD ou clés SSH (Settings → CI/CD → Variables). Si accès à un autre projet, accorder les permissions (l'utilisateur doit être membre ou une [autorisation de lecture sur le projet](#) sinon le clone échoue).
 - *GitHub Actions* : Vérifier que les **Secrets** sont définis au niveau repository/organisation. Utiliser `secrets.GITHUB_TOKEN` ou un **Personal Access Token** pour les opérations API. En cas de workflows sur forks, activer « **Actions** » pour le dépôt forké.
 - *Azure DevOps* : Contrôler les **Service connections** pour les dépôts externes. Si message `requires permission to access resource`, aller dans le portail Azure DevOps pour *autoriser* explicitement le pipeline (ex : pool d'agents, repo Git cible). Pour TFVC ou Git dans autre projet, désactiver *Limit job authorization scope to current project* si besoin (sinon checkout bloqué).
- **Exemples** : Jenkins pipeline Groovy :

groovy

Copy

```

pipeline {
  agent any
  stages {
    stage('Checkout'){
      steps {
        git url: 'git@github.com:orga/projet.git', credentialsId: 'ssh-github'
      }
    }
  }
}

```

Azure Pipelines YAML :

yaml

Copy

resources:

repositories:

- repository: ToolsRepo

type: git

name: OtherProject/Tools

endpoint: MyAzureServiceConnection

S'assurer d'avoir autorisé ce pipeline à accéder au service connection.

1.2 Intégration SCM / Checkout

- **Symptôme** : Échec du clone du dépôt (ex. fatal: repository not found, access denied), ou code non mis à jour.
- **Causes probables** : Mauvaise URL/BRANCHE, proxy ou SSL manquant, suppression de branche, problème réseau. Dans GitLab, règles only:changes mal configurées peuvent déclencher des pipelines inattendus ou bloquer. Azure Pipelines peut refuser l'accès si *Limit job authorization scope* est activé pour un repo hors projet.

- **Diagnostic :** Examiner les logs de checkout (console Jenkins ou log GitLab CI). Tester manuellement l'URL en ligne de commande sur l'agent. Vérifier la syntaxe du fichier pipeline (indentation YAML, commandes checkout).
- **Solutions :**
 - *Jenkins* : Vérifier Git Publisher ou git plugin. Si host SSH, ajouter le dépôt aux hôtes connus de l'agent Jenkins.
 - *GitLab* : Confirmer que l'URL du repo est correcte (namespace/projet). Pour des règles rules: changes, noter que p.ex. un nouveau branche sans PR évalue toujours changes comme vrai.
 - *GitHub* : Utiliser l'action officielle : uses: actions/checkout@v3 dans la workflow. Activer *submodules* si nécessaire.
 - *Azure* : Cas fréquent : projet différent du pipeline. Dans ce cas, aller dans *Project Settings* → *Repositories* pour vérifier que le pipeline a bien l'autorisation de checkout (et désactiver *Limit job authorization scope* si le repo est dans un autre projet).

1.3 Échecs de build (Compilation/Tests)

- **Symptôme :** Échec du job de build (compilation ratée, tests plantés, timeouts).
- **Causes probables :** Code erroné, dépendances manquantes, VM agent incomplète. Différences d'environnement (versions de JDK, Node.js, etc.). Problème de plateforme (par ex. CRLF vs LF, sensible sur certains scripts).
- **Diagnostic :** Consulter les logs complets du build. Reproduire localement sur une machine équivalente (même image container ou OS). Vérifier les versions de langages/outils. Dans Jenkins, activer le mode *quiet output* ou *verbose* selon besoin.
- **Solutions :**
 - Installer les dépendances requises sur l'agent ou via un conteneur Docker.
 - Pour les problèmes Java/Node, préciser des versions fixes (ex. Maven/Gradle -Djava.security.egd parfois pour les entropies).
 - Ajuster les scripts pour gérer les différences de fin de ligne. Exemple Jenkins (Windows) : ajoutez `\r\n` ou utilisez `sh 'dos2unix script.sh'`.
 - *Azure DevOps* note aussi des erreurs aléatoires de MSBuild ou de fichier occupé : il recommande, si besoin, d'exécuter la même commande en local pour isoler le problème.

1.4 Dépendances & Cache

- **Symptôme** : Téléchargement constant des dépendances, timeout réseau, cache corrompu ou obsolète.
- **Causes probables** : Pas ou mauvaise configuration du cache. Registre Maven/NPM inaccessible. Conflits de versions (cache non invalidé).
- **Diagnostic** : Vérifier les logs de téléchargement (ex. npm install / mvn package). Comprendre si le cache est utilisé (messages “cached” ou pas).
- **Solutions** :
 - Configurer le cache sur la plateforme : *GitLab* cache:key (voir la [doc GitLab cache](#)). *GitHub Actions* avec l'action actions/cache (établir une clé stable, p.ex. \${{ runner.os }}-npm-\${{ hashFiles('**/package-lock.json') }}). *Azure Pipelines* propose la tâche CacheBeta@1 ou Cache@2 (préciser path et key).
 - Purger les caches corrompus (parfois renommer manuellement pour forcer le refill).

1.5 Dérive d'environnement / Configuration

- **Symptôme** : Comportement incohérent entre stages ou environnements (dev vs prod). Variables non prises en compte.
- **Causes probables** : Paramètres d'environnement définis différemment (scope utilisateur vs root), chemins relatifs erronés, absence de variables dans tel environnement.
- **Diagnostic** : Comparer les valeurs de variables (env) dans chaque job. Utiliser echo ou printenv dans le script. Dans Jenkins, voir *Build Environment*. Dans Kubernetes, comparer ConfigMaps/Secrets appliqués.
- **Solutions** :
 - Centraliser la configuration (fichier YAML, ConfigMaps, .env).
 - Utiliser une plateforme comme HashiCorp Vault pour injecter dynamiquement les secrets/variables.
 - Vérifier les contextes/clusters actifs (kubectl config).

1.6 Conteneurs / Images Docker

- **Symptôme** : Erreurs liées à Docker : build échoué, push refusé, ou déploiement Kubernetes échoue (Crash, ImagePull).

- **Causes probables** : Dockerfile incorrect, image introuvable, tag absent, quotas, registry inaccessible.
- **Diagnostic** : Tester localement le docker build et docker run. Dans CI : consulter la sortie du docker build (peut être très verbeux). Si push, s’assurer des droits et du bon repository/tag.
- **Solutions** :
 - Valider le nom complet de l’image (registry/nom:tag). Assurer que l’image est *poussée* avant déploiement.
 - Si docker login est nécessaire (image privée), configurer le job pour le faire.
 - Exemple Azure Pipelines :

yaml

Copy

- task: Docker@2

inputs:

containerRegistry: 'MyDockerHub'

repository: 'moncompte/monimage'

command: buildAndPush

tags: '\$(Build.BuildId)'

- *Déploiement Kubernetes* : voir sections 2.2 CrashLoopBackOff et 2.3 ImagePullBackOff ci-dessous.

1.7 Stockage d’artefacts

- **Symptôme** : Artéfacts de build non enregistrés ou corrompus (erreur “path not found”).
- **Causes probables** : Mauvais chemin (glob patterns incorrects), limite de taille dépassée, espace disque insuffisant sur serveur.
- **Diagnostic** : Dans Jenkins, vérifier dans *Archived Artifacts*. GitLab affiche “Expired” si délai dépassé. Azure Pipelines signale en général l’erreur précise.
- **Solutions** :
 - Spécifier correctement les chemins. Ex. Jenkins : archiveArtifacts artifacts: '**/target/*.jar'.

- Allouer plus de rétention ou espace (docs Azure mentionnent quotas Pipeline Artifacts).
- Utiliser un dépôt d'artefacts externe (Nexus, Artifactory, GitLab Package Registry, Azure Artifacts).

1.8 Runners / Agents / Executors

- **Symptôme** : Jobs en attente (runner offline, agent désengagé), exécution très lente, “TaskCanceled” / self-hosted runner lost communication.
- **Causes probables** : Agents désactivés, labels mal configurés, limite de capacité dépassée. (P.ex. GitLab : runner uniquement disponible pour certains tags, ou Azure : pas d'agents disponibles).
- **Diagnostic** : Vérifier le statut des agents :
 - *Jenkins* : Dashboard nodes/agents. Logs du master et de l'agent (voir jenkins.log).
 - *GitLab* : **Admin Area** → **Runners** ou dans les paramètres du projet, voir si runner actif.
 - *GitHub Actions* : **Actions** → **Runners** (self-hosted) pour voir l'état.
 - *Azure* : **Project Settings** → **Agents pools**.
- **Solutions** :
 - Redémarrer ou reconnecter l'agent. Vérifier la communication réseau (firewall).
 - Pour GitLab, s'assurer que executor (docker/shell) est valide, et que le runner est autorisé pour ce projet. Un runner “online” mais sans jobs peut signifier un **tag mismatch** entre le job et le runner configuré.
 - Augmenter le nombre d'agents ou utiliser des agents hébergés/cloud selon la charge.

1.9 Syntaxe / Configuration du pipeline

- **Symptôme** : Échec immédiat avec message de syntaxe invalide, ou étapes ignorées. Exemple : gitlab-ci.yml syntax invalid, Jenkinsfile error.
- **Causes probables** : Fichier YAML mal indenté, caractère non UTF8, séquence invalide. Jenkinsfile Groovy avec erreurs.
- **Diagnostic** :
 - *GitLab* : Utiliser l'outil de **Lint** intégré (CI/CD → Pipelines → Lint).

- *GitHub Actions* : GitHub signale l'erreur de parsing sur la page run.
- *Azure* : Le portail ou logs du job affichent souvent la ligne en erreur.
- *Jenkins* : Ouvrir la vue « Replay » ou logs du build pour voir où Groovy échoue.
- **Solutions** : Corriger la syntaxe. Astuce : commencer avec un exemple minimal fonctionnel de pipeline, puis ajouter des parties.

1.10 Gestion des secrets

- **Symptôme** : Accès refusé à ressources externes (API, registry, cloud) en raison de clés manquantes; ou pire, fuite de secrets dans les logs.
- **Causes probables** : Secrets codés en dur, absence de chiffrement dans CI, secrets non masqués (***) .
- **Diagnostic** : Scanner les logs pour password/token ou utiliser des outils (GitLeaks, etc.). Vérifier qu'aucune variable de secret n'est affichée en clair.
- **Solutions** :
 - Stocker les secrets dans les mécanismes dédiés : *Jenkins Credentials*, *GitLab CI Variables (masked)*, *GitHub Actions Secrets*, *Azure Key Vault/Variables de pipeline*.
 - Ne pas passer les secrets en arguments de commandes en clair.
 - Utiliser les fonctionnalités de liaison (ex : `withCredentials, ${ secrets.MY_SECRET }`).
 - **Meilleures pratiques** : Variables à durée de vie courte, gestion centralisée (Vault), scan régulier. Le blog DevOps.com souligne que l'échec de gestion des secrets dans CI/CD est devenu un vecteur de compromission de la chaîne logistique logicielle (par ex. injection de clés dans des pipelines tiers).

1.11 Réseau / Timeouts

- **Symptôme** : Jobs qui bloquent ou échouent sur des opérations réseau : timeout, connection refused, échecs DNS, ETIMEDOUT.
- **Causes probables** : Problèmes de connectivité vers Git (blocage d'IP), serveurs distants (registry, API) lents ou inaccessibles, proxy mal configuré.
- **Diagnostic** : Exécuter des pings, nslookup, ou curl sur l'agent. Vérifier si les adresses IP sortantes sont autorisées (par ex. certains clusters cloud limitent le

trafic). Dans GitHub, on peut activer les logs de réseau du runner (Github-hosted ou self-hosted) en mode verbose (tel que GIT_CURL_VERBOSE).

- **Solutions :**

- S'assurer que les agents possèdent la configuration proxy appropriée.
- Augmenter les timeouts si nécessaire (par ex. `git config --global http.lowSpeedLimit 0` etc.).
- *Azure DevOps* propose d'activer les logs verboses et de vérifier les sorties.
- Mettre en place des backoffs et tentatives (Many actions CI ont retry possible).

1.12 Limites de ressources

- **Symptôme :** Jobs tués (OOMKilled), throttling CPU, mauvaise performance.
Kubernetes : pods terminés, OOMKilled.
- **Causes probables :** Ressources non définies ou insuffisantes. Absence de limites sur les containers. Overcommit de la machine/cluster.
- **Diagnostic :** Sur les runners, surveiller l'utilisation (Jenkins : *Manage* -> *System Log*, GitLab Runner monitoring, Azure Pipeline logs). Sur Kubernetes, `kubectl describe pod` montre souvent OOMKilled. Les logs systèmes (`dmesg/syslog`) peuvent indiquer un OOM killer.
- **Solutions :**
 - **Jenkins/GitLab/GitHub/Azure :** Spécifier les ressources si possible (certains runners cloud le permettent). S'assurer que la machine a au moins les ressources minimales (ex. 4 GB RAM pour builds Java).
 - **Kubernetes :** Définir `resources.requests/limits` dans les pods pour éviter la surconsommation. Utiliser des jobs de type *pipeline* plus légers, ou scaler horizontalement.
 - *Azure :* Checker la section « Job time-out » (exemple : augmenter la limite de timeout dans les paramètres de pipeline si un job a besoin de plus de 60 minutes).

1.13 Tests instables / Conditions de course

- **Symptôme :** Échecs aléatoires de tests unitaire/intégration, exécutions répétables donnent des résultats différents.

- **Causes probables** : Tests dépendant de l'ordre ou d'états partagés, ressources non isolées (DB partagée sans nettoyage), accès concurrent sur un même artefact.
- **Diagnostic** : Exécuter les tests en boucle (for i in {1..100}; do pytest; done) pour repérer un pattern. Revoir les logs pour *flaky test*.
- **Solutions** :
 - Isoler les tests (bases de données temporaires ou mocking).
 - Autoriser des échecs intermittents tolérés (p.ex. GitLab allow_failure, ou Jira).
 - Exécuter les tests séquentiellement plutôt que parallèlement si le problème vient de la concurrence.

1.14 Sécurité CI/CD et incidents récents

- **Symptôme** : Comportement anormal du pipeline (ex. exfiltration de données, job qui télécharge des packages suspects, pipelines externe compromis).
- **Causes probables** : Composants tiers (actions, plugins) compromis, clés volées.
- **Diagnostic** : Scanner les logs de pipeline pour requêtes réseau inattendues (C2C), vérifier l'intégrité des dépendances (hashes). Contrôler les versions des actions/outils utilisés.
- **Solutions** :
 - Appliquer le principe du moindre privilège aux tokens et secrets de pipeline (tokens GitHub Actions passent désormais par OIDC pour éviter les clés statiques).
 - Utiliser des listes blanches/blacklists sur les actions ou plugins (GitHub permet de restreindre les actions utilisées).
 - Mettre à jour régulièrement les outils CI (Jenkins, GitLab Runner, actions).
 - **Cas pratique** : En mars 2026, plusieurs attaques de chaîne d'approvisionnement ont compromis des outils de scanning (Trivy, KICS) dans GitHub Actions. Les versions malveillantes véhiculaient un malware de vol de credentials dans chaque pipeline affecté. Solution préventive : utiliser des versions signées/vérifiées d'actions, rejeter les versions outre passables (via policy), et limiter les permissions de l'environnement d'exécution (E.g. intervalles de réseau).

- Scanner en continu pour les vulnérabilités et secrets (ex. GitLab Secret Detection, GitHub Advanced Security).

2. Déploiement & Kubernetes

2.1 Stratégies de déploiement et rollback

- **Blue/Green et Canary** : Permettent de déployer en graduel. En cas d'échec, le rollback dépend de la stratégie :
 - *Blue/Green* : basculer manuellement ou automatiquement vers l'ancienne version (ex. switch DNS/service).
 - *Canary* : réduire la part de trafic envoyée à la nouvelle version ou annuler le déploiement incrémental.
- **Rollback** : Kubernetes et Azure Pipelines fournissent des commandes dédiées. Exemple K8s :

bash

Copy

```
kubectl rollout undo deployment/mon-application
```

ou spécifier une révision :

bash

Copy

```
kubectl rollout undo deployment/mon-application --to-revision=3
```

(Voir également Azure Pipelines **Undo deployments** ou Jenkins avec plugins de pipeline déploiement). Toujours vérifier qu'une révision antérieure fonctionne avant rollback.

2.2 CrashLoopBackOff (Kubernetes)

- **Symptôme** : kubectl get pods affiche CrashLoopBackOff pour un pod (statut Waiting, réinstallations répétées).
- **Causes probables** : L'application dans le conteneur plante à son démarrage (erreur d'exécution, dépendance manquante, segmentation fault). Une livenessProbe trop stricte peut aussi entraîner un crash infini.
- **Diagnostic** :
 1. kubectl describe pod <pod> : la section *State* montre CrashLoopBackOff, *Last State* indique souvent Error.

2. Examiner les *Events* en fin de description : recherche de Back-off restarting failed container.
3. Récupérer les logs du container :

css

Copy

```
kubectl logs <pod> -c <container> --previous
```

pour voir la raison de l'erreur avant redémarrage.

- **Solutions :**

- Corriger l'erreur dans l'application (dépendance manquante, mauvaise commande, fail fast).
- Allouer plus de ressources si OOM ; ou augmenter les délais de *initialDelaySeconds* dans les probes si l'app met du temps à démarrer.
- Déployer un conteneur debug (image de débogage) pour investiguer si l'app ne sort pas de l'Entrypoint.

- **Bonnes pratiques :** Toujours

configurer *readinessProbe* et *livenessProbe* adaptées. Comme le recommande CloudBees, si un container n'est tout simplement pas prêt au démarrage, augmenter le délai initial du probe peut éviter des redémarrages précoces.

2.3 ImagePullBackOff (Kubernetes)

- **Symptôme :** Pod reste indéfiniment en état ImagePullBackOff, généralement après avoir affiché ErrImagePull. Signifie que Kubernetes n'arrive pas à récupérer l'image.
- **Causes probables :** Référence d'image incorrecte (tag inexistant, repository introuvable), absence de credentials pour un registry privé, problèmes réseau (DNS/proxy), ou limites de débit (Docker Hub rate-limits).
- **Diagnostic :**
 1. `kubectl describe pod <pod>` → section *Events* : lire l'erreur, par ex. Manifest not found, no pull access, unauthorized, TLS handshake timeout, etc.
 2. Selon Komodor, on opère ainsi :
 - Si l'événement indique Manifest ... not found ou unknown, c'est probablement un tag manquant (corriger le tag de l'image).

- Si no pull access ou unauthorized, c'est un problème de credentials (vérifier imagePullSecrets ou droits du compte de service).
- En cas de i/o timeout ou TLS handshake timeout, c'est un problème réseau/DNS (vérifier la connectivité vers le registre).

- **Solutions :**

- S'assurer du chemin complet de l'image (hostname/namespace/nom:tag) dans le déploiement. Par exemple :

yaml

Copy

spec:

containers:

- name: app

image: monrepo.azurecr.io/monimage:v1.2.3

imagePullSecrets:

- name: acr-secret

- Créer et référencer un Secret Kubernetes de type docker-registry si privé :

bash

Copy

kubectl create secret docker-registry acr-secret \

--docker-server=monrepo.azurecr.io \

--docker-username=<utilisateur> \

--docker-password=<motdepasse> \

--docker-email=<email>

- Configurer correctement la politique imagePullPolicy. En environnement contrôlé, on peut changer IfNotPresent ou pré-puller l'image.
- Mettre en place un miroir local ou authentifié pour les registries publics (ex. mirror DockerHub pour éviter le rate-limit).

2.4 Probes de liveness/readiness

- **Symptôme** : Kubernetes tue les conteneurs de façon répétée (Restarting) ou les considère non « prêts » (pas d'IP de service). Les logs montrent *probe failed* (status 503 ou *connection refused*).
- **Causes probables** : Point de terminaison de santé (/healthz, etc.) incorrect, port mal renseigné, le conteneur répond trop lentement. Une application bloquée ou démarrant lentement fait échouer le probe.
- **Diagnostic** :
 - `kubectl describe pod <pod>` : ligne Liveness ou Readiness échecs.
 - `kubectl port-forward` vers le pod puis `curl localhost:<port>/health` pour tester manuellement le probe.
- **Solutions** :
 - Vérifier les *commandes/path* des probes dans le YAML du déploiement. Par ex. `livenessProbe.httpGet.path` doit correspondre à un endpoint valide dans l'application.
 - Allonger `initialDelaySeconds` ou `timeoutSeconds`. Par exemple, CloudBees recommande d'augmenter le *Health Check Initial Delay* si l'API met du temps à démarrer.
 - S'assurer que l'application est réellement opérationnelle (parfois le probe pointe vers le login ou autre page restreinte).
 - Si l'application est lente sous forte charge, revoir ses performances ou passer à des probes TCP/exec le temps de debug.

2.5 RBAC (Kubernetes)

- **Symptôme** : `kubectl` renvoie *Forbidden* pour une opération : ex. Error from server (Forbidden): deployments.apps is forbidden...
- **Causes probables** : Le ServiceAccount utilisé n'a pas la permission (Role/ClusterRole) sur la ressource/cible. Cela peut toucher aussi les volumes PV/PVC ou ConfigMaps.
- **Diagnostic** :
 - Le message d'erreur RBAC de Kubernetes est explicite sur la permission manquante, p.ex. :

vbnet

Copy

Error from server (Forbidden): pods is forbidden: User "system:serviceaccount:prod:my-sa" cannot list resource "pods"...

Ce message indique quelle ressource et quel verbe (list) manquent.

- Utiliser `kubectl auth can-i <verb> <resource> --as=system:serviceaccount:... pour tester`. Par exemple :

csharp

Copy

```
kubectl auth can-i list pods --as=system:serviceaccount:production:app-sa
```

comme montré dans la doc OneUptime.

- **Solutions :**

- Créer ou ajuster un Role/ClusterRole avec les droits nécessaires, puis un RoleBinding/ClusterRoleBinding liant ce rôle au ServiceAccount ou utilisateur.
- En DevOps, éviter de donner aux pods un token avec trop de droits. Préférer des comptes de service dédiés par application.

2.6 Ingress (Kubernetes)

- **Symptôme :** L'application n'est pas joignable via l'Ingress (404 ou 503).
- **Causes probables :** Règles *host* ou *path* mal configurées, secret TLS incorrect, le service cible inexistant ou port erroné, Ingress Controller mal déployé.
- **Diagnostic :**
 - `kubectl describe ingress <nom>` : vérifier les règles et événements.
 - `kubectl get ingress` montre le *ADDRESS* (IP ou domaine).
 - Vérifier les logs de l'Ingress Controller (ex. NGINX) : `kubectl logs -n ingress-nginx <controller-pod>`.
 - Tester directement le service backend (sans ingress) pour isoler l'erreur.
- **Solutions :** Comme le souligne PrometheusTech, vérifier :
 - Que le champ *host* de l'Ingress YAML correspond au domaine utilisé (ex. `myapp.example.com`).
 - Que le `tls.secretName` existe et correspond au certificat du *host*.
 - Que `service.name` et `service.port.number` pointent bien vers le service Kubernetes exposant les pods.

- Exemple d’Ingress YAML corrigé (authentique) :

yaml

Copy

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app-ingress
  annotations:
    kubernetes.io/ingress.class: nginx
spec:
  tls:
  - hosts:
    - myapp.example.com
    secretName: myapp-tls-secret
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
      backend:
        service:
          name: my-app-service
          port:
            number: 80
```

Ce correctif (aligner host et secretName) a résolu le 404 dans l’exemple.

2.7 Autres incidents (Kubernetes)

- **NodeNotReady** : vérifier le statut du cluster (kubectl get nodes). Redémarrer kubelet ou vérifier kubelet logs si un nœud dysfonctionne.
- **Ingress Controller** : s'assurer qu'il est correctement déployé (pod non CrashLoop, configMap valide).
- **Services non accessibles** : vérifier les Endpoint : kubectl get endpoints <service> pour voir si les pods backend sont enregistrés.

3. Comparaison multi-plateformes

Aspect / Plateforme	Jenkins	GitLab CI	GitHub Actions	Azure Pipelines
Définition Pipeline	Jenkinsfile (Groovy). Déclaratif ou script.	.gitlab-ci.yml (YAML). Multi-stage.	Fichiers YAML dans .github/workflows/.	azure-pipelines.yml (YAML).
Agents/Runners	Master/Agents (JNLP, SSH). Labels assignés.	Runners partagés ou privés, <i>tags</i> optionnels.	Hosted (GitHub) ou self-hosted runners.	Agents Microsoft-hosted ou self-hosted.
Checkout SCM	Par défaut via checkout scm ou étape Git.	Automatique; git clone inclus. Permissions GitLab.	Via actions/checkout@v3. Clé SSH ou token.	Automatique pour Azure Repos; pour GitHub/
Secrets/Variables	Credentials Plugin (mot de passe, SSH, etc.).	Variables CI/CD masquées (interface).	Secrets par repo/org; \$ secrets.NAME.	Variables cachées, groupes de variables, con
Artifacts/Cache	archiveArtifacts et plugin <i>Copy To</i>	Clé artifacts: et cache: dans le YAML.	Actions intégrées (actions/upload-artifact, actions/cache).	Tâches PublishPipelineArtifact et CacheBeta.
Debug logs	Activer -D dans Java, <i>Verbose</i> pour plugins, ou Jenkins CLI -p.	Utiliser trace: true dans .gitlab-ci.yml, ou activer <i>debug</i> .	Variables ACTIONS_RUNNER_DEBUG, ACTIONS_STEP_DEBUG, GIT_TRACE=1.	Ajouter system.debug: true dans variables po